University of Exeter

ECM2433: The C Family

Lecture 07: Generics and Traits

The duplication problem

```rust
fn largest_i32(list: &[i32]) -> Option<&i32> {
    if list.is_empty() { return None; }
    let mut largest = &list[0];
    for item in &list[1..] {
        if item > largest { largest = item; }
    }
    Some(largest)
}
```

```rust
fn largest_char(list: &[char]) -> Option<&char> {
    if list.is_empty() { return None; }
    let mut largest = &list[0];
    for item in &list[1..] {
        if item > largest { largest = item; }
    }
    Some(largest)
}
```

- Two functions with identical logic, differing only in type
- C++ solves this with templates: `template<typename T>`
- Rust generics are similar: write the logic once, compiler generates specialised versions
- Key difference: Rust requires you to declare what `T` can do (trait bounds, covered later)
- [Book ch 10.1]

University of Exeter

Generic functions

```rust
fn largest<T: PartialOrd>(list: &[T]) -> Option<&T> {
    if list.is_empty() { return None; }
    let mut largest = &list[0];
    for item in &list[1..] {
        if item > largest { largest = item; }
    }
    Some(largest)
}

let nums = vec![34, 50, 25, 100, 65];
let chars = vec!['y', 'm', 'a', 'q'];

let result = largest(&nums);  // Some(&100)
let result = largest(&chars); // Some(&'y')
```

- `<T: PartialOrd>`: `T` is a type parameter that must support comparison
- One function works for both `i32` and `char`
- The compiler generates a specialised version for each type used
- [Book ch 10.1]

University of Exeter

Generic structs

```
1 struct Point<T> {x: T, y: T}
2
3 let int_point = Point { x: 5, y: 10 };
4 let float_point = Point { x: 1.0, y: 4.0 };
5 let mixed = Point { x: 5, y: 4.0 }; // error: mismatched types
6
7 struct Pair<T, U> {first: T, second: U}
8
9 let pair = Pair { first: 1, second: "hello" }; // T=i32, U=&str
```

- `Point<T>`: both fields share one type parameter
- Mixing types with a single parameter results in a compile error
- `Pair<T, U>`: two type parameters allow different types
- Standard library examples: `Result<T, E>`, `HashMap<K, V>`
- [Book ch 10.1]

University of Exeter

Generic methods

```rust
struct Point<T> {x: T, y: T}

impl<T> Point<T> {
    fn new(x: T, y: T) -> Self {
        Point { x, y }
    }
}

let p = Point::new(5, 10);
println!("{}", p.x); // 5
```

- `impl<T>` declares `T` for the whole block
- `Self` refers to `Point<T>`
- Methods and associated functions work exactly as in non-generic structs
- The `<T>` on `impl` is required so Rust knows `T` is a type parameter, not a concrete type
- [Book ch 10.1]

University of Exeter

What are traits?

```rust
fn largest<T: PartialOrd>(list: &[T]) -> Option<&T> { ... }

let chars = vec!['y', 'm', 'a', 'q'];
let result = largest(&chars); // Some('y')
```

- A trait defines behaviour that types can share
- `PartialOrd` is a trait: it guarantees the `>` operator exists
- Like C++ abstract classes or Java interfaces: "any type that has these methods"
- Traits connect to generics: constrain `T` by requiring specific traits
- Rust checks generic function bodies against declared bounds, and checks concrete types against bounds at use sites
- [Book ch 10.2]

University of Exeter

Defining and implementing a trait

```rust
trait Shape {
    fn area(&self) -> f64;
}

struct Circle {
    radius: f64,
}

struct Rectangle {
    width: f64,
    height: f64,
}

impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}

impl Shape for Rectangle {
    fn area(&self) -> f64 {
        self.width * self.height
    }
}
```

- `trait Shape`: defines a method signature that types must implement
- `impl Shape for Circle`: provides the implementation for a specific type
- Each type has its own implementation of the same trait
- [Book ch 10.2]

Default method implementations

```rust
trait Shape {
    fn area(&self) -> f64;

    fn describe(&self) -> String {
        format!("a shape with area {:.2}", self.area())
    }
}

// Circle: only implements area(), inherits describe()
impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}

// Rectangle: overrides describe() with a custom version
impl Shape for Rectangle {
    fn area(&self) -> f64 { self.width * self.height }

    fn describe(&self) -> String {
        format!("{}x{} rectangle", self.width, self.height)
    }
}
```

- A trait method with a body is a default implementation
- Types inherit the default unless they override it
- Default methods can call other trait methods
- [Book ch 10.2]

Trait bounds

```rust
use std::fmt::Display;

fn print_largest<T: PartialOrd + Display>(list: &[T]) {
    if list.is_empty() { return; }
    let mut largest = &list[0];
    for item in &list[1..] {
        if item > largest { largest = item; }
    }
    println!("the largest is {largest}");
}

print_largest(&[34, 50, 25, 100]); // the largest is 100
print_largest(&['y', 'm', 'a']);   // the largest is y
```

- `T: PartialOrd`: `T` must support comparison
- `T: Display`: `T` must support formatting with `{ }`
- `T: PartialOrd + Display`: combine bounds with `+`
- The compiler rejects any type that does not satisfy all bounds
- [Book ch 10.2]

University of Exeter

Where clauses

```rust
// Inline bounds: gets crowded with multiple parameters
fn process<T: PartialOrd + Display, U: Clone + Display>(t: T, u: U) { ... }

// Same thing with a where clause
fn process<T, U>(t: T, u: U)
where
    T: PartialOrd + Display,
    U: Clone + Display,
{
    ...
}
```

- `where` moves trait bounds after the parameter list
- Identical meaning: just a different layout
- Preferred when bounds are long or involve multiple parameters
- [Book ch 10.2]

University of Exeter

Derive macros

```rust
#[derive(Debug, Clone, PartialEq)]
struct Point {x: f64, y: f64}

let p1 = Point { x: 1.0, y: 2.0 };
let p2 = p1.clone();

println!("{:?}", p1);       // Point { x: 1.0, y: 2.0 }
assert_eq!(p1, p2);         // passes: PartialEq
```

- `#[derive(...)]`: automatically implements traits for your type
- `Debug`: enables `{:?}` formatting (for printing and debugging)
- `Clone`: enables `.clone()` to create an explicit copy
- `PartialEq`: enables `==` and `!=` comparison
- `assert_eq!` requires both `Debug` and `PartialEq`

University of Exeter

- [Appendix C: Derivable Traits]

Common derives reference

- `Debug`: `{:?}` formatting for printing and debugging
- `Clone`: `.clone()` for explicit copies (semantics depend on the type)
- `PartialEq`: `==` and `!=` comparison
- `Eq`: full equality (required for `HashMap` keys, usually paired with `PartialEq`)
- `Hash`: hashing (also required for `HashMap` keys)
- `Default`: provides a default value via `Type::default()`
- Rule of thumb: start with `#[derive(Debug)]`
- [Appendix C: Derivable Traits]

University of Exeter

Demo

University of Exeter

Recap

- Generics: write code once with `<T>`, compiler generates specialised versions
- Traits: define shared behaviour; `impl Trait for Type` provides implementations
- Trait bounds: `T: PartialOrd + Display` constrains what types a generic accepts
- `where` clauses: move bounds after the parameter list for readability
- Derive macros: `#[derive(Debug, Clone, PartialEq)]` for automatic trait implementations
- Required reading: Book ch 10.1 (generics), 10.2 (traits), Appendix C (derives)

University of Exeter